

Ch07 문자열 검색 · Ch08 리스트 · Ch09 트리

문제만 있는 타이핑 정리 버전

Ch07 문자열 검색

[보기선택형 문제]

1. 문자열 검색의 설명으로 가장 옳은 것은?
 - ① 숫자 데이터를 정렬하는 과정
 - ② 문자열 안에서 특정 패턴을 찾는 과정
 - ③ 트리 구조를 생성하는 과정
 - ④ 연결 리스트를 탐색하는 과정
2. 문자열 검색에서 검색되는 대상 문자열은?
 - ① 패턴
 - ② 키
 - ③ 텍스트
 - ④ 노드
3. 문자열 검색에서 찾으려는 문자열은?
 - ① 패턴
 - ② 텍스트
 - ③ 루트
 - ④ 인덱스
4. 브루트 포스법의 특징으로 가장 적절한 것은?
 - ① 해시 기반 검색
 - ② 가장 단순한 문자열 검색 방법
 - ③ 트리 기반 검색
 - ④ 정렬이 반드시 필요
5. 브루트 포스법에서 불일치가 발생 시 처리방법은?
 - ① 패턴을 여러 칸 건너뛰
 - ② 패턴을 한 칸 오른쪽으로 이동
 - ③ 텍스트를 역순으로 비교
 - ④ 탐색 종료
6. 브루트 포스법의 시간복잡도로 가장 적절한 것은?
 - ① $O(1)$
 - ② $O(\log n)$
 - ③ $O(n)$
 - ④ $O(nm)$

7. 브루트 포스법이 비효율적인 이유는?

- ① 재귀 사용
- ② 비교 결과를 활용하지 않음
- ③ 트리 구조 필요
- ④ 추가 배열 사용

8. KMP법의 핵심 아이디어는?

- ① 힙 사용
- ② 비교 결과 재활용
- ③ 무작위 탐색
- ④ 트리 균형 유지

9. KMP법에서 사용하는 표는?

- ① 힙 배열
- ② 해시 테이블
- ③ skip table
- ④ adjacency matrix

10. KMP법이 장점으로 가장 적절한 것은?

- ① 구현이 가장 단순함
- ② 비교를 반복하지 않음
- ③ 이미 비교한 정보를 활용함
- ④ 항상 $O(1)$ 보장

11. KMP법에서 불일치가 발생하면 활용하는 것은?

- ① 압축
- ② skip table
- ③ 큐 구조
- ④ 재귀 호출

12. KMP에서 패턴의 일부가 이미 일치했다면?

- ① 처음부터 다시 비교
- ② 일치 정보를 활용하여 건너뛸
- ③ 탐색 종료
- ④ 정렬 수행

13. KMP법의 시간복잡도는?

- ① $O(1)$
- ② $O(\log n)$
- ③ $O(n + m)$
- ④ $O(n^2)$

14. KMP법에서 skip table을 만드는 목적은?

- ① 메모리 절약
- ② 중복 비교 제거
- ③ 트리 생성
- ④ 데이터 정렬

15. 보이어-무어법의 특징으로 옳은 것은?

- ① 패턴의 앞 문자부터 비교
- ② 패턴의 끝 문자부터 비교
- ③ 항상 한 칸씩 이동
- ④ 재귀 기반 탐색

16. 보이어-무어법이 빠른 이유는?

- ① 비교 없이 탐색
- ② 여러 칸 건너뛰기 가능
- ③ 항상 정렬 수행
- ④ 재귀 깊이 감소

17. 다음 중 문자열 검색 알고리즘이 아닌 것은?

- ① 브루트 포스법
- ② KMP법
- ③ 보이어-무어법
- ④ 병합 정렬

18. KMP법에서 패턴 내부의 겹치는 문자열 정보를 사용하는 이유는?

- ① 메모리 절약
- ② 비교 위치를 효율적으로 이동
- ③ 정렬 수행
- ④ 트리 구성

19. 보이어-무어법은 일반적으로 어떤 특징이 있는가?

- ① 실제 수행 속도가 빠름
- ② 항상 구현이 가장 간단함
- ③ 추가 메모리를 사용하지 않음
- ④ 트리 구조 필요

20. 브루트 포스법과 비교했을 때 KMP법의 차이점은?

- ① 문자열 정렬 수행
- ② 비교 결과를 재사용
- ③ 배열 대신 트리 사용
- ④ 항상 재귀 호출 사용

[O/X 문제]

21. 문자열 검색은 문자열 안에서 특정 패턴을 찾는 과정이다. (O/X)
22. 문자열 검색에서 찾으려는 문자열을 텍스트(text)라고 한다. (O/X)
23. 브루트 포스법은 문자열 검색 알고리즘 중 가장 단순한 방법이다. (O/X)
24. 브루트 포스법은 불일치가 발생하면 패턴을 한 칸 이동하여 다시 비교한다. (O/X)
25. 브루트 포스법은 이미 비교한 결과를 적극적으로 재사용한다. (O/X)
26. 브루트 포스법의 최악 시간복잡도는 $O(nm)$ 이다. (O/X)
27. KMP법은 패턴 내부의 일치 정보를 활용한다. (O/X)
28. KMP법은 불일치가 발생하면 항상 처음부터 다시 비교한다. (O/X)
29. KMP법은 skip table을 사용한다. (O/X)
30. KMP법의 시간복잡도는 일반적으로 $O(n + m)$ 이다. (O/X)
31. KMP법은 브루트 포스법보다 불필요한 비교를 줄일 수 있다. (O/X)
32. 보이어-무어법은 패턴의 앞 문자부터 비교를 시작한다. (O/X)
33. 보이어-무어법은 일반적으로 실제 수행 속도가 빠른 편이다. (O/X)
34. 보이어-무어법은 여러 문자를 한 번에 건너뛸 수 있다. (O/X)
35. 문자열 검색 알고리즘은 항상 정렬 과정이 필요하다. (O/X)
36. 이진 검색은 문자열 검색 알고리즘의 대표적인 예이다. (O/X)
37. KMP법은 이미 비교한 문자열 정보를 재활용한다. (O/X)
38. 보이어-무어법은 불일치 발생 시 큰 폭으로 이동할 수 있다. (O/X)
39. 브루트 포스법은 구현이 단순하다는 장점이 있다. (O/X)
40. 문자열 검색 알고리즘은 텍스트 안에서 원하는 패턴의 존재 여부를 검사할 수 있다. (O/X)

[보기선택형 문제]

41. 배열 기반 리스트의 특징으로 가장 적절한 것은?

- ① 삽입·삭제 시 데이터 이동이 거의 없다
- ② 인덱스를 이용한 접근이 빠르다
- ③ 포인터를 반드시 사용한다
- ④ 메모리를 비연속적으로 사용한다

42. 연결 리스트의 장점으로 가장 적절한 것은?

- ① 인덱스 접근이 빠름
- ② 메모리 사용이 매우 적음
- ③ 삽입·삭제 시 데이터 이동이 적음
- ④ 항상 정렬 상태 유지

43. 단순 연결 리스트에서 마지막 노드를 삭제할 때 비효율적인 이유는?

- ① 재귀 호출이 필요
- ② 루트 노드가 없음
- ③ 이전 노드를 바로 알 수 없음
- ④ 포인터 사용 불가

44. 단순 연결 리스트의 노드 클래스에서 일반적으로 저장되는 정보로 가장 적절한 것은?

- ① 데이터와 next 포인터
- ② 데이터와 힙 주소
- ③ 인덱스 번호와 부모 노드
- ④ 루트 주소만 저장

45. 연결 리스트에서 predecessor node의 의미는?

- ① 현재 노드의 부모 노드
- ② 현재 노드의 자식 노드
- ③ 현재 노드의 바로 앞 노드
- ④ 현재 노드의 마지막 노드

46. 연결 리스트에서 successor node의 의미는?

- ① 현재 노드의 바로 뒤 노드
- ② 현재 노드의 부모 노드
- ③ 현재 노드의 리프 노드
- ④ 현재 노드의 루트 노드

47. 연결 리스트에서 특정 노드를 탐색하는 기본 방식은?

- ① 이진 탐색
- ② 순차 탐색
- ③ 해시 탐색
- ④ 트리 탐색

48. 연결 리스트가 배열보다 중간 삽입에 유리한 이유는?

- ① 정렬 상태 유지 가능
- ② 인덱스 접근 가능
- ③ 데이터 이동이 거의 필요 없음
- ④ 메모리를 적게 사용함

49. 다음 중 연결 리스트의 단점으로 가장 적절한 것은?

- ① 삽입 불가능
- ② 임의 접근이 어려움
- ③ 데이터 저장 불가능
- ④ 포인터 사용 불가

50. 단순 연결 리스트 구현 코드에서 마지막 노드의 next 값으로 가장 적절한 것은?

- ① 자기 자신
- ② head
- ③ None
- ④ predecessor

51. 연결 리스트 구현에서 current 포인터의 역할로 가장 적절한 것은?

- ① 리스트 전체 길이 저장
- ② 현재 처리 중인 노드를 참조
- ③ 마지막 노드 삭제
- ④ head 노드 생성

52. 단순 연결 리스트의 remove_first() 함수 수행 결과로 가장 적절한 것은?

- ① tail이 삭제됨
- ② head가 이전 노드를 가리킴
- ③ head가 기존 head.next로 변경됨
- ④ 리스트 전체 초기화

53. 커서를 이용한 연결 리스트 구현의 특징으로 가장 적절한 것은?

- ① 포인터 대신 배열 인덱스를 사용
- ② 항상 완전 리스트로 구현
- ③ 삭제 연산 지원 불가
- ④ 트리 구조 저장 전용

54. 커서 연결 리스트 구현에서 free list의 역할은?

- ① 트리 높이 저장
- ② 삭제된 레코드 재사용 관리
- ③ 정렬 상태 유지
- ④ 재귀 호출 처리

55. 커서 연결 리스트 구현에서 삭제 연산이 반복되면 발생할 수 있는 문제는?

- ① 루트 노드 충돌
- ② 빈 레코드 증가
- ③ 이진 탐색 불가능
- ④ 배열 접근 불가능

56. 원형 연결 리스트 구현의 특징으로 옳은 것은?

- ① 마지막 노드의 next가 None
- ② 마지막 노드가 첫 노드를 가리킴
- ③ 항상 prev 포인터 사용
- ④ head 노드가 존재하지 않음

57. 이중 연결 리스트 구현에서 각 노드가 저장하는 정보로 가장 적절한 것은?

- ① next 포인터만 저장
- ② prev 포인터만 저장
- ③ prev와 next 포인터 모두 저장
- ④ 인덱스 번호만 저장

58. 이중 연결 리스트의 장점으로 가장 적절한 것은?

- ① 메모리 사용량 감소
- ② 양방향 탐색 가능
- ③ 항상 $O(1)$ 검색 가능
- ④ 정렬 속도 향상

59. 원형 이중 연결 리스트 구현의 설명으로 가장 적절한 것은?

- ① 배열과 트리의 결합 구조
- ② 원형 리스트와 이중 연결 리스트를 결합한 구조
- ③ 큐와 스택을 결합한 구조
- ④ 힙 기반 저장 구조

60. 연결 리스트에서 특정 데이터를 탐색하는 시간복잡도로 가장 적절한 것은?

- ① $O(1)$
- ② $O(\log n)$
- ③ $O(n)$
- ④ $O(n^2)$

[O/X 문제]

- 61. 연결 리스트에서 각 노드는 데이터와 다음 노드를 가리키는 포인터를 가진다. (O/X)
- 62. 단순 연결 리스트에서 predecessor node를 바로 찾기 위해서는 일반적으로 head부터 탐색해야 한다. (O/X)
- 63. 연결 리스트의 tail node의 next 값은 일반적으로 자기 자신을 가리킨다. (O/X)
- 64. 배열 기반 리스트는 중간 삽입 시 데이터 이동이 발생할 수 있다. (O/X)
- 65. 연결 리스트는 삽입·삭제 시 데이터 이동이 거의 필요하지 않다. (O/X)
- 66. 연결 리스트에서 특정 데이터를 검색할 때 일반적으로 이진 검색(binary search)을 사용한다. (O/X)
- 67. 연결 리스트의 search() 함수는 보통 head 노드부터 순차적으로 탐색한다. (O/X)
- 68. 연결 리스트가 비어 있으면 head 값은 None이 된다. (O/X)
- 69. 연결 리스트에서 노드가 1개뿐이라면 그 노드는 head node이면서 tail node이다. (O/X)
- 70. remove_first 수행 시 head는 기존 head.next를 참조하도록 변경된다. (O/X)
- 71. remove_last는 단순 연결 리스트에서 tail 바로 이전 노드를 찾기 위해 순차 탐색이 필요할 수 있다. (O/X)
- 72. current 포인터는 현재 주목하고 있는 노드를 가리키는 데 사용된다. (O/X)
- 73. next() 함수는 current를 current.next로 이동시키는 역할을 한다. (O/X)
- 74. 연결 리스트의 모든 노드를 삭제하는 clear() 함수는 보통 tail부터 삭제를 반복한다. (O/X)
- 75. 커서를 이용한 연결 리스트에서는 포인터 대신 배열 인덱스를 사용한다. (O/X)
- 76. 커서 연결 리스트에서 삭제가 반복되면 빈 레코드가 증가할 수 있다. (O/X)
- 77. 프리 리스트(free list)는 삭제된 레코드를 재사용하기 위한 구조이다. (O/X)
- 78. 원형 리스트에서는 tail node의 next가 head node를 가리킨다. (O/X)
- 79. 이중 연결 리스트에서는 각 노드가 prev와 next 포인터를 모두 가진다. (O/X)
- 80. 원형 이중 연결 리스트의 더미 노드는 삽입·삭제 처리를 단순화하는 데 도움을 준다. (O/X)

Ch09 트리

[보기선택형 문제]

81. 다음 중 트리(tree)의 특징으로 옳지 않은 것은?

- ① 루트 노드는 하나만 존재한다
- ② 노드는 여러 자식을 가질 수 있다
- ③ 모든 노드는 반드시 부모를 가진다
- ④ 트리는 계층적 구조를 표현한다

82. 트리의 높이(height)에 대한 설명으로 가장 적절한 것은?

- ① 전체 노드 수
- ② 루트의 차수
- ③ 루트에서 가장 먼 리프까지의 거리
- ④ 리프 노드 개수

83. 다음 중 깊이 우선 탐색(DFS)에 해당하지 않는 것은?

- ① preorder
- ② inorder
- ③ postorder
- ④ level order

84. 중위 순회(inorder)를 수행했을 때 오름차순 결과를 얻을 수 있는 자료구조는?

- ① 일반 트리
- ② 힙
- ③ 이진 검색 트리
- ④ 큐

85. 다음 중 이진 검색 트리(BST)의 조건으로 옳은 것은?

- ① 왼쪽 서브트리의 모든 키는 부모보다 크다
- ② 오른쪽 서브트리의 모든 키는 부모보다 작다
- ③ 왼쪽은 작고 오른쪽은 크다
- ④ 부모와 자식의 값은 항상 같다

86. 다음 중 BST 탐색 과정으로 옳은 것은?

- ① 현재 노드와 비교 후 양쪽 모두 탐색
- ② 항상 왼쪽만 탐색
- ③ 비교 결과에 따라 한 방향만 탐색
- ④ 리프부터 탐색 시작

87. 균형 잡힌 BST에서 탐색 시간복잡도로 가장 적절한 것은?

- ① $O(1)$
- ② $O(\log n)$
- ③ $O(n)$
- ④ $O(n^2)$

88. 다음 값을 순서대로 BST에 삽입할 때 루트 노드는?

[8, 3, 10, 1]

- ① 1
- ② 3
- ③ 8
- ④ 10

89. 다음 값을 BST에 순서대로 삽입했을 때 6의 부모 노드는?

[8, 3, 10, 1, 6, 14]

- ① 1
- ② 3
- ③ 8
- ④ 10

90. 다음 BST에서 key=14 탐색 시 비교 횟수는?

그림: root 8, 왼쪽 3, 오른쪽 10, 10의 오른쪽 14

- ① 1
- ② 2
- ③ 3
- ④ 4

91. 다음 중 BST에서 가장 비효율적인 경우는?

- ① 균형 잡힌 상태
- ② 무작위 삽입
- ③ 한쪽으로 치우친 상태
- ④ 완전 이진 트리 형태

92. 다음 데이터를 순서대로 BST에 삽입할 때 트리 높이가 가장 높아질 가능성이 높은 것은?

- ① 5, 3, 7, 1, 9
- ② 1, 2, 3, 4, 5
- ③ 4, 2, 6, 1, 3
- ④ 8, 4, 10, 2, 6

93. BST를 중위 순회했을 때 결과가 다음과 같다면 루트가 될 수 있는 값은?

[1, 3, 5, 7, 9]

- ① 1
- ② 5
- ③ 9
- ④ 모두 가능

94. BST에서 key=7을 탐색할 때 올바른 이동 순서는?

그림: root 10, 왼쪽 5, 5의 오른쪽 8, 8의 왼쪽 7

- ① 10 → 5 → 8 → 7
- ② 10 → 8 → 7
- ③ 5 → 8 → 7
- ④ 10 → 5 → 7

95. BST에서 삽입(insert)의 특징으로 가장 적절한 것은?

- ① 항상 루트에 삽입
- ② 정렬 후 삽입
- ③ 조건을 만족하는 위치를 찾아 삽입
- ④ 리프에만 삽입 불가

96. 다음 중 BST 삭제 과정에서 가장 복잡한 경우는?

- ① 자식이 없는 노드 삭제
- ② 자식이 1개인 노드 삭제
- ③ 자식이 2개인 노드 삭제
- ④ 루트가 아닌 노드 삭제

97. BST에서 자식이 2개인 노드를 삭제할 때 일반적으로 사용하는 방법은?

- ① 임의 노드와 교환
- ② 왼쪽 서브트리 최대값 또는 오른쪽 서브트리 최소값 사용
- ③ 항상 루트 삭제
- ④ 모든 노드 재정렬

98. 다음 중 완전 이진 트리의 특징으로 가장 적절한 것은?

- ① 마지막 레벨은 아무 순서로 채워짐
- ② 노드는 오른쪽부터 채워짐
- ③ 왼쪽부터 차례대로 채워짐
- ④ 모든 노드가 반드시 두 자식을 가짐

99. 완전 이진 트리를 배열로 표현할 때 부모 인덱스가 i 이면 왼쪽 자식 인덱스는?

- ① $i + 1$
- ② $2i$
- ③ $2i + 1$
- ④ $2i + 2$

100. 다음 중 BST의 장점으로 가장 적절한 것은?

- ① 항상 $O(1)$ 탐색 가능
- ② 중위 순회 시 정렬 결과 획득 가능
- ③ 메모리를 사용하지 않음
- ④ 모든 경우 $O(\log n)$ 보장

[O/X 문제]

101. 트리에서 루트(root) 노드는 하나만 존재한다. (O/X)

102. 리프(leaf) 노드는 자식 노드가 존재하지 않는 노드이다. (O/X)

103. 형제(sibling) 노드는 부모가 같은 노드를 의미한다. (O/X)

104. 트리의 높이(height)는 전체 노드의 개수를 의미한다. (O/X)

105. BFS는 낮은 레벨부터 차례대로 탐색하는 방법이다. (O/X)

106. DFS는 리프까지 우선 내려가는 방식의 탐색이다. (O/X)

107. preorder 순회는 현재 노드를 먼저 방문한다. (O/X)

108. inorder 순회는 왼쪽 → 노드 → 오른쪽 순서로 방문한다. (O/X)

109. postorder 순회는 현재 노드를 가장 마지막에 방문한다. (O/X)

110. 이진 트리는 각 노드가 최대 두 개의 자식을 가질 수 있다. (O/X)

111. 완전 이진 트리는 마지막 레벨까지 반드시 모두 채워져 있어야 한다. (O/X)

112. 완전 이진 트리는 같은 레벨에서 왼쪽부터 차례대로 노드가 채워진다. (O/X)

113. 이진 검색 트리(BST)에서는 왼쪽 서브트리의 모든 키값이 부모보다 작아야 한다. (O/X)

114. 이진 검색 트리(BST)에서는 오른쪽 서브트리의 모든 키값이 부모보다 커야 한다. (O/X)

115. 이진 검색 트리를 중위 순회하면 키값이 오름차순으로 출력된다. (O/X)

116. BST 탐색에서는 비교 결과에 따라 한 방향의 서브트리만 탐색한다. (O/X)

117. 균형 잡힌 BST의 탐색 시간복잡도는 일반적으로 $O(\log n)$ 이다. (O/X)

118. BST가 한쪽으로 치우치면 탐색 시간복잡도는 $O(n)$ 이 될 수 있다. (O/X)

119. BST에서 자식 노드가 2개인 노드를 삭제하는 경우가 가장 복잡하다. (O/X)

120. BST에서 자식이 2개인 노드를 삭제할 때 일반적으로 왼쪽 서브트리의 최대값 또는 오른쪽 서브트리의 최소값을 사용한다. (O/X)